数据结构基础

算法实现讲义

课程核心算法的 C 语言实现 + 详细中文注释

涵盖第 2–9 章：算法分析 / 线性表 / 树 / 堆 / 排序 / 散列 / 并查集 / 图

每个算法给出**标准 C 实现**与**逐行中文注释**，并标注**时间 / 空间复杂度**。
代码风格与课程（Weiss 教材 / PTA）一致，可直接对照理解与上机。

2026 年 6 月 24 日

目录

阅读说明 **2**

1 算法分析（Algorithm Analysis） **3**
1.1 最大子序列和 Maximum Subsequence Sum . . . 3
1.2 二分查找 Binary Search . . . 4
1.3 欧几里得算法 Euclid's Algorithm（最大公约数） . . . 5
1.4 快速幂 Exponentiation by Squaring . . . 5

2 线性表、栈与队列（Lists, Stacks & Queues） **7**
2.1 单链表 Singly Linked List . . . 7
2.2 栈 Stack（LIFO） . . . 8
2.3 循环队列 Circular Queue（FIFO） . . . 8
2.4 栈的应用 . . . 9
2.5 多项式加法 Polynomial Addition（链表） . . . 11

3 二叉树与搜索树（Binary Trees & Search Trees） **12**
3.1 二叉树结构与遍历 Traversals . . . 12
3.2 二叉搜索树 Binary Search Tree . . . 13
3.3 表达式树 Expression Tree . . . 15
3.4 线段树 Segment Tree（区间和） . . . 15

4 优先队列与二叉堆（Priority Queue / Binary Heap） **17**

5 排序（Sorting） **19**

6 散列表（Hashing） **25**
6.1 分离链接法 Separate Chaining . . . 25
6.2 开放地址法 Open Addressing（平方探测 Quadratic Probing） . . . 25

7 并查集（Disjoint Set / Union-Find） **28**

8 图（Graph Algorithms） **30**
8.1 拓扑排序 Topological Sort . . . 30
8.2 无权最短路 Unweighted Shortest Path（BFS） . . . 31
8.3 Dijkstra 单源最短路（非负权） . . . 31
8.4 SPFA 单源最短路（可含负权） . . . 32
8.5 深度优先搜索 DFS 与连通分量 . . . 33
8.6 最小生成树 MST：Prim 算法 . . . 34
8.7 最小生成树 MST：Kruskal 算法 . . . 34
8.8 网络最大流 Maximum Flow（Edmonds–Karp） . . . 36

阅读说明

- 代码用 **C 语言**，命名与课件 / 教材一致；注释为中文，便于理解每一步**为什么**这样写。
- 每个算法前的灰色文字标注**复杂度**（T= 时间，S= 空间）。蓝框为补充说明，红框为易错点。
- 结构体定义在每章开头给出一次，后续算法默认沿用。
- 复杂度记号：N 为规模，V/E 为图的顶点 / 边数，h 为树高。

1 算法分析（Algorithm Analysis）

1.1 最大子序列和 Maximum Subsequence Sum

求序列中连续子段的最大和（全负时返回 0）。同一问题、四种复杂度，是“算法改进”的经典案例。

算法一：三重循环（暴力枚举）

$T = O(N^3),\ S = O(1)$

```
int MaxSubSum1(const int A[], int N) {
    int MaxSum = 0;                          /* 最终答案，全负时保持 0 */
    for (int i = 0; i < N; i++)              /* 枚举子段起点 i */
        for (int j = i; j < N; j++) {        /* 枚举子段终点 j */
            int ThisSum = 0;
            for (int k = i; k <= j; k++)     /* 重新累加 A[i..j] —— 多余的重复计算 */
                ThisSum += A[k];
            if (ThisSum > MaxSum)            /* 更新最大值 */
                MaxSum = ThisSum;
        }
    return MaxSum;
}
```

算法二：二重循环（递增维护当前和）

$T = O(N^2),\ S = O(1)$

```
int MaxSubSum2(const int A[], int N) {
    int MaxSum = 0;
    for (int i = 0; i < N; i++) {            /* 固定起点 i */
        int ThisSum = 0;
        for (int j = i; j < N; j++) {        /* 终点 j 右移时复用上一次的和 */
            ThisSum += A[j];                 /* 关键：把内层第三重循环省掉 */
            if (ThisSum > MaxSum)
                MaxSum = ThisSum;
        }
    }
    return MaxSum;
}
```

算法三：分治 Divide and Conquer

$T = O(N \log N),\ S = O(\log N)$

```
/* 最大子段和只有三种情况：完全在左半、完全在右半、跨越中点 */
int MaxSubSum3(const int A[], int Left, int Right) {
    if (Left == Right)                       /* 递归边界：单个元素 */
        return A[Left] > 0 ? A[Left] : 0;

    int Center = (Left + Right) / 2;
    int MaxLeft  = MaxSubSum3(A, Left, Center);       /* 左半最大和 */
```

```
    int MaxRight = MaxSubSum3(A, Center + 1, Right);  /* 右半最大和 */

    /* 求跨越中点的最大和：从中点向左、向右各扩展一次 */
    int MaxLeftBorder = 0, LeftBorderSum = 0;
    for (int i = Center; i >= Left; i--) {            /* 含中点向左的最大后缀和 */
        LeftBorderSum += A[i];
        if (LeftBorderSum > MaxLeftBorder) MaxLeftBorder = LeftBorderSum;
    }
    int MaxRightBorder = 0, RightBorderSum = 0;
    for (int i = Center + 1; i <= Right; i++) {       /* 中点右侧的最大前缀和 */
        RightBorderSum += A[i];
        if (RightBorderSum > MaxRightBorder) MaxRightBorder = RightBorderSum;
    }
    int MaxCross = MaxLeftBorder + MaxRightBorder;    /* 跨中点最大和 */

    /* 三者取最大 */
    int m = MaxLeft > MaxRight ? MaxLeft : MaxRight;
    return m > MaxCross ? m : MaxCross;
}
/* 初始调用：MaxSubSum3(A, 0, N-1); 递推式 T(N)=2T(N/2)+O(N) */
```

算法四：在线算法 On-line

$T = O(N),\ S = O(1)$

```
int MaxSubSum4(const int A[], int N) {
    int ThisSum = 0, MaxSum = 0;          /* ThisSum: 当前后缀和 */
    for (int j = 0; j < N; j++) {
        ThisSum += A[j];                  /* 累加当前元素 */
        if (ThisSum > MaxSum)
            MaxSum = ThisSum;             /* 出现更大的和则更新答案 */
        else if (ThisSum < 0)
            ThisSum = 0;                  /* 前缀和为负，对后面只会拖累，直接丢弃归零 */
    }
    return MaxSum;                        /* 只扫一遍；任意时刻都持有当前正确答案 */
}
```

1.2　二分查找 Binary Search

在有序数组中查找 X

$T = O(\log N),\ S = O(1)$

```
/* 前提：A[0..N-1] 已按升序排列（静态数据）。找到返回下标，否则返回 -1 */
int BinarySearch(const int A[], int X, int N) {
    int Low = 0, High = N - 1;
    while (Low <= High) {                 /* 区间非空时继续 */
        int Mid = Low + (High - Low) / 2; /* 防止 (Low+High) 溢出 */
        if (A[Mid] < X)
```

```
            Low = Mid + 1;                  /* 目标在右半 */
        else if (A[Mid] > X)
            High = Mid - 1;                 /* 目标在左半 */
        else
            return Mid;                     /* 命中 */
    }
    return -1;                              /* 区间为空仍未找到 */
}
```

1.3　欧几里得算法 Euclid's Algorithm（最大公约数）

原理：$\gcd(m,n)=\gcd(n,\ m \bmod n)$。每步问题规模快速下降，$T=O(\log\min(m,n))$。

迭代版

$T=O(\log\min(M,N))$，$S=O(1)$

```
unsigned int Gcd(unsigned int M, unsigned int N) {
    while (N > 0) {                         /* N 变成 0 时，M 即为答案 */
        unsigned int Rem = M % N;           /* 余数 */
        M = N;                              /* 用 (N, M%N) 替换 (M, N) */
        N = Rem;
    }
    return M;                               /* gcd(M, 0) = M */
}
```

递归版

$T=O(\log\min(M,N))$，$S=O(\log\min(M,N))$（递归栈）

```
unsigned int GcdR(unsigned int M, unsigned int N) {
    if (N == 0) return M;                   /* 递归边界 */
    return GcdR(N, M % N);                  /* 尾递归，可被编译器优化为循环 */
}
```

1.4　快速幂 Exponentiation by Squaring

计算 X^N。利用 $X^N=(X^{N/2})^2$（偶）或 $(X^{(N-1)/2})^2\cdot X$（奇），每步指数减半。

迭代版（推荐）

$T=O(\log N)$，$S=O(1)$

```
long long Pow(long long X, unsigned int N) {
    long long Result = 1;
    while (N > 0) {
        if (N & 1)              /* 当前二进制最低位为 1，则把对应的 X 的幂乘入结果 */
            Result *= X;
        X *= X;                 /* X 自乘：X -> X^2 -> X^4 -> ... */
        N >>= 1;                /* 指数右移一位（除以 2） */
    }
    return Result;
```

```
}
/* 工程中常用「取模快速幂」：每次乘法后对模数取余，防溢出 */
```

递归版

$T = O(\log N),\ S = O(\log N)$

```
long long PowR(long long X, unsigned int N) {
    if (N == 0) return 1;
    long long half = PowR(X, N / 2);      /* 先算 X^(N/2)，只递归一次 */
    if (N % 2 == 0) return half * half;   /* 偶数：平方 */
    else            return half * half * X; /* 奇数：平方后再乘一个 X */
}
```

2 线性表、栈与队列（Lists, Stacks & Queues）

2.1 单链表 Singly Linked List

结构定义与基本操作

插入 / 删除（已知位置）$O(1)$，查找 $O(N)$

```c
typedef struct Node *PtrToNode;
struct Node {
    int Element;          /* 数据域 */
    PtrToNode Next;       /* 指针域：指向后继结点 */
};
typedef PtrToNode List;   /* 带头结点(dummy head)的链表，由头结点指针表示 */

/* 在结点 P 之后插入值 X（已知 P，故 O(1)） */
void Insert(int X, PtrToNode P) {
    PtrToNode Tmp = (PtrToNode)malloc(sizeof(struct Node));
    Tmp->Element = X;
    Tmp->Next = P->Next;  /* 新结点先接住 P 原来的后继（顺序不能反！） */
    P->Next = Tmp;        /* 再让 P 指向新结点 */
}

/* 删除链表 L 中第一个值为 X 的结点 */
void Delete(int X, List L) {
    PtrToNode P = L;                        /* 从头结点出发，便于拿到「前驱」 */
    while (P->Next != NULL && P->Next->Element != X)
        P = P->Next;                        /* 停在待删结点的前驱处 */
    if (P->Next != NULL) {                  /* 找到了 */
        PtrToNode Tmp = P->Next;
        P->Next = Tmp->Next;                /* 前驱跨过待删结点 */
        free(Tmp);                          /* 释放空间 */
    }
}
```

链表反转 Reverse（高频考点）

$T = O(N)$，$S = O(1)$

```c
/* 反转不带头结点的链表，返回新表头 */
List Reverse(List Head) {
    PtrToNode Prev = NULL, Cur = Head;
    while (Cur != NULL) {
        PtrToNode Next = Cur->Next;         /* 先存住后继，防止断链 */
        Cur->Next = Prev;                   /* 当前结点指针反向，指向前驱 */
        Prev = Cur;                         /* Prev、Cur 各前进一步 */
        Cur = Next;
    }
    return Prev;                            /* 原表尾成为新表头 */
```

```
}
```

2.2　栈 Stack（LIFO）

数组实现

Push / Pop / Top 均 $O(1)$

```
struct StackRecord {
    int Capacity;          /* 容量 */
    int TopOfStack;        /* 栈顶下标，空栈约定为 -1 */
    int *Array;            /* 存放元素的数组 */
};
typedef struct StackRecord *Stack;

int IsEmpty(Stack S) { return S->TopOfStack == -1; }
int IsFull (Stack S) { return S->TopOfStack == S->Capacity - 1; }

void Push(int X, Stack S) {
    if (IsFull(S)) return;                 /* 满栈不再压入（实现错误，需检查）*/
    S->Array[++S->TopOfStack] = X;         /* 先自增栈顶，再写入 */
}
int Top(Stack S) {
    if (IsEmpty(S)) return -1;             /* 空栈取顶是 ADT 错误 */
    return S->Array[S->TopOfStack];
}
void Pop(Stack S) {
    if (IsEmpty(S)) return;                /* 空栈弹出是 ADT 错误 */
    S->TopOfStack--;                       /* 栈顶下移即可 */
}
```

2.3　循环队列 Circular Queue（FIFO）

数组实现（用 Size 字段区分满/空）

Enqueue / Dequeue 均 $O(1)$

```
struct QueueRecord {
    int Capacity;
    int Front;             /* 队头下标 */
    int Rear;              /* 队尾下标 */
    int Size;              /* 当前元素个数：用它区分「空」和「满」，可用满全部格子 */
    int *Array;
};
typedef struct QueueRecord *Queue;

/* 下标回绕：到末尾后回到 0 */
static int Succ(int X, Queue Q) { return (X + 1) % Q->Capacity; }
```

```
12
13 void Enqueue(int X, Queue Q) {
14     if (Q->Size == Q->Capacity) return;    /* 队满 */
15     Q->Size++;
16     Q->Rear = Succ(Q->Rear, Q);            /* 队尾后移（循环）*/
17     Q->Array[Q->Rear] = X;
18 }
19 int Dequeue(Queue Q) {                     /* 出队并返回队头元素 */
20     if (Q->Size == 0) return -1;           /* 队空 */
21     Q->Size--;
22     int X = Q->Array[Q->Front];
23     Q->Front = Succ(Q->Front, Q);          /* 队头后移（循环）*/
24     return X;
25 }
```

注意

若**不用** Size 字段、只用 Front 与 Rear，必须**空出一格**来区分满 / 空（满：(Rear+1)%Cap==Front），此时最大容量为 Capacity-1。

2.4　栈的应用

1. 括号匹配 Balancing Symbols

$T = O(N)$，在线算法

```
1  /* 检查 () [] {} 是否匹配。返回 1 表示匹配成功 */
2  int BalanceCheck(const char *s) {
3      char stack[1000]; int top = -1;
4      for (int i = 0; s[i]; i++) {
5          char c = s[i];
6          if (c == '(' || c == '[' || c == '{')
7              stack[++top] = c;              /* 左括号入栈 */
8          else if (c == ')' || c == ']' || c == '}') {
9              if (top == -1) return 0;       /* 右括号来时栈空 -> 不匹配 */
10             char L = stack[top--];         /* 弹出栈顶左括号 */
11             if ((c==')'&&L!='(') || (c==']'&&L!='[') || (c=='}'&&L!='{'))
12                 return 0;                  /* 类型不匹配 */
13         }
14     }
15     return top == -1;                      /* 扫描完栈应为空 */
16 }
```

2. 后缀表达式求值 Postfix Evaluation

$T = O(N)$

```
1  /* 操作数入栈；遇运算符弹出两个操作数计算后压回 */
2  int EvalPostfix(const char *expr) {
```

```
    int stack[1000]; int top = -1;
    for (int i = 0; expr[i]; i++) {
        char c = expr[i];
        if (c >= '0' && c <= '9') {
            stack[++top] = c - '0';       /* 数字字符转为整数入栈 */
        } else if (c=='+'||c=='-'||c=='*'||c=='/') {
            int b = stack[top--];         /* 注意：先弹出的是右操作数 b */
            int a = stack[top--];         /* 后弹出的是左操作数 a */
            int r = 0;
            switch (c) {
                case '+': r = a + b; break;
                case '-': r = a - b; break;
                case '*': r = a * b; break;
                case '/': r = a / b; break;
            }
            stack[++top] = r;             /* 结果压回栈 */
        }
    }
    return stack[top];                    /* 最终栈顶即为答案 */
}
```

3. 中缀转后缀 Infix to Postfix

$T = O(N)$

```
/* 优先级：* / 高于 + - ；遇到优先级不低于当前运算符的栈顶先弹出 */
int Prec(char op) { return (op=='+'||op=='-') ? 1 : (op=='*'||op=='/') ? 2 : 0; }

void InfixToPostfix(const char *in, char *out) {
    char stack[1000]; int top = -1, k = 0;
    for (int i = 0; in[i]; i++) {
        char c = in[i];
        if (c >= '0' && c <= '9') {
            out[k++] = c;                 /* 操作数直接输出 */
        } else if (c == '(') {
            stack[++top] = c;             /* 左括号直接入栈 */
        } else if (c == ')') {
            while (top != -1 && stack[top] != '(')
                out[k++] = stack[top--];  /* 弹出直到遇左括号 */
            if (top != -1) top--;         /* 丢弃左括号 */
        } else {                          /* 运算符 */
            while (top != -1 && Prec(stack[top]) >= Prec(c))
                out[k++] = stack[top--];  /* 弹出不低于当前优先级的栈顶 */
            stack[++top] = c;             /* 当前运算符入栈 */
        }
    }
    while (top != -1) out[k++] = stack[top--]; /* 输出剩余运算符 */
```

```
    out[k] = '\0';
}
```

2.5　多项式加法 Polynomial Addition（链表）

两个按指数降序的多项式相加

$T = O(N_1 + N_2)$

```
typedef struct PolyNode *Poly;
struct PolyNode { int coef; int expon; Poly next; };  /* 系数、指数、后继 */

/* 尾插一个新项到结果链表，rear 为当前尾指针 */
static void Attach(int c, int e, Poly *pRear) {
    Poly t = (Poly)malloc(sizeof(struct PolyNode));
    t->coef = c; t->expon = e; t->next = NULL;
    (*pRear)->next = t; *pRear = t;       /* 接到尾后，更新尾指针 */
}

Poly PolyAdd(Poly a, Poly b) {
    Poly head = (Poly)malloc(sizeof(struct PolyNode)); /* 哑头结点 */
    Poly rear = head;
    while (a && b) {                      /* 双指针归并，按指数从高到低 */
        if (a->expon > b->expon) { Attach(a->coef, a->expon, &rear); a = a->next; }
        else if (a->expon < b->expon) { Attach(b->coef, b->expon, &rear); b = b->next;
     }
        else {                            /* 指数相同：系数相加 */
            int sum = a->coef + b->coef;
            if (sum != 0) Attach(sum, a->expon, &rear); /* 系数为 0 的项不保留 */
            a = a->next; b = b->next;
        }
    }
    for (; a; a = a->next) Attach(a->coef, a->expon, &rear); /* 接上剩余项 */
    for (; b; b = b->next) Attach(b->coef, b->expon, &rear);
    rear->next = NULL;
    Poly t = head; head = head->next; free(t); /* 去掉哑头结点 */
    return head;
}
```

3 二叉树与搜索树（Binary Trees & Search Trees）

3.1 二叉树结构与遍历 Traversals

结构定义

```c
typedef struct TreeNode *Tree;
struct TreeNode {
    int Element;
    Tree Left;            /* 左孩子 */
    Tree Right;           /* 右孩子 */
};
```

前序 / 中序 / 后序遍历（递归）

$T = O(N),\ S = O(h)$（递归栈）

```c
void PreOrder(Tree T) {               /* 前序：根 -> 左 -> 右 */
    if (T == NULL) return;
    visit(T->Element);                /* 先访问根 */
    PreOrder(T->Left);
    PreOrder(T->Right);
}
void InOrder(Tree T) {                /* 中序：左 -> 根 -> 右（BST 中序得升序）*/
    if (T == NULL) return;
    InOrder(T->Left);
    visit(T->Element);                /* 在两次递归之间访问根 */
    InOrder(T->Right);
}
void PostOrder(Tree T) {              /* 后序：左 -> 右 -> 根 */
    if (T == NULL) return;
    PostOrder(T->Left);
    PostOrder(T->Right);
    visit(T->Element);               /* 最后访问根 */
}
```

中序遍历（迭代，用栈）

$T = O(N),\ S = O(h)$

```c
void InOrderIter(Tree T) {
    Tree stack[1000]; int top = -1;
    while (T != NULL || top != -1) {
        while (T != NULL) {           /* 一路向左，沿途结点入栈 */
            stack[++top] = T;
            T = T->Left;
        }
        T = stack[top--];             /* 弹出最左结点 */
        visit(T->Element);            /* 访问它 */
        T = T->Right;                 /* 转向其右子树，重复 */
```

```
    }
}
```

层序遍历 Level-order（用队列，类似 BFS）

$T = O(N),\ S = O(N)$

```
void LevelOrder(Tree T) {
    if (T == NULL) return;
    Tree queue[1000]; int front = 0, rear = 0;
    queue[rear++] = T;                /* 根入队 */
    while (front < rear) {
        Tree node = queue[front++];   /* 出队 */
        visit(node->Element);
        if (node->Left)  queue[rear++] = node->Left;   /* 左孩子入队 */
        if (node->Right) queue[rear++] = node->Right;  /* 右孩子入队 */
    }
}
```

由中序 + 后序重建二叉树

$T = O(N^2)$（每次线性找根）

```
/* in[inL..inR] 中序, post[postL..postR] 后序, 返回子树根 */
Tree Build(int in[], int post[], int inL, int inR, int postL, int postR) {
    if (inL > inR) return NULL;                          /* 空子树 */
    Tree root = (Tree)malloc(sizeof(struct TreeNode));
    root->Element = post[postR];                         /* 后序最后一个是根 */
    int k = inL;
    while (in[k] != post[postR]) k++;                    /* 在中序中定位根 */
    int leftLen = k - inL;                               /* 左子树结点个数 */
    root->Left  = Build(in, post, inL, k-1, postL, postL+leftLen-1);
    root->Right = Build(in, post, k+1, inR, postL+leftLen, postR-1);
    return root;
}
/* 已知 中序+前序 或 中序+后序 可唯一重建；仅前序+后序不行 */
```

3.2　二叉搜索树 Binary Search Tree

查找 / 最小 / 最大

$T = O(h)$；平衡时 $O(\log N)$，退化时 $O(N)$

```
Tree Find(int X, Tree T) {
    if (T == NULL) return NULL;        /* 没找到 */
    if (X < T->Element) return Find(X, T->Left);   /* 比根小，去左子树 */
    if (X > T->Element) return Find(X, T->Right);  /* 比根大，去右子树 */
    return T;                          /* 相等，命中 */
}
Tree FindMin(Tree T) {                 /* 最小值：一直向左走 */
    if (T == NULL) return NULL;
```

```
    while (T->Left != NULL) T = T->Left;
    return T;
}
Tree FindMax(Tree T) {                  /* 最大值：一直向右走 */
    if (T == NULL) return NULL;
    while (T->Right != NULL) T = T->Right;
    return T;
}
```

插入 Insert

$T = O(h)$

```
Tree Insert(int X, Tree T) {
    if (T == NULL) {                    /* 到达空位 -> 在此新建结点 */
        T = (Tree)malloc(sizeof(struct TreeNode));
        T->Element = X; T->Left = T->Right = NULL;
    } else if (X < T->Element)
        T->Left  = Insert(X, T->Left);   /* 递归插入左子树，并接回 */
    else if (X > T->Element)
        T->Right = Insert(X, T->Right);  /* 递归插入右子树，并接回 */
    /* X 已存在则什么也不做（不允许重复键） */
    return T;                           /* 切记返回 T! */
}
```

删除 Delete（三种情况）

$T = O(h)$

```
Tree Delete(int X, Tree T) {
    if (T == NULL) return NULL;         /* 没找到，直接返回 */
    if (X < T->Element)
        T->Left = Delete(X, T->Left);
    else if (X > T->Element)
        T->Right = Delete(X, T->Right);
    else if (T->Left && T->Right) {     /* 情况3：左右孩子都在 */
        Tree Min = FindMin(T->Right); /* 用右子树最小结点顶替 */
        T->Element = Min->Element;
        T->Right = Delete(Min->Element, T->Right); /* 再删掉那个最小结点 */
    } else {                            /* 情况1/2：至多一个孩子 */
        Tree Tmp = T;
        if (T->Left == NULL) T = T->Right;  /* 用唯一孩子（或 NULL）顶替 */
        else                 T = T->Left;
        free(Tmp);
    }
    return T;
}
```

3.3　表达式树 Expression Tree

由后缀表达式构建

$T = O(N)$

```
/* 遇操作数：建叶子入栈；遇运算符：弹两棵子树合成新树入栈 */
Tree BuildExprTree(const char *postfix) {
    Tree stack[1000]; int top = -1;
    for (int i = 0; postfix[i]; i++) {
        char c = postfix[i];
        Tree t = (Tree)malloc(sizeof(struct TreeNode));
        t->Element = c; t->Left = t->Right = NULL;
        if (c=='+'||c=='-'||c=='*'||c=='/') {
            t->Right = stack[top--];  /* 先弹出的作右子树 */
            t->Left  = stack[top--];  /* 后弹出的作左子树 */
        }
        stack[++top] = t;             /* 新树入栈 */
    }
    return stack[top];                /* 栈中唯一的树即表达式树 */
}
/* 对该树做中/后/前序遍历，分别得到 中缀/后缀/前缀 表达式 */
```

3.4　线段树 Segment Tree（区间和）

建树 / 区间查询 / 单点更新

建树 $O(N)$；查询、更新 $O(\log N)$

```
int tree[4 * MAXN];                   /* 通常开 4N 大小以容纳所有结点 */
int A[MAXN];                          /* 原数组 */

/* 建树：node 表示当前结点编号，负责区间 [start, end] */
void Build(int node, int start, int end) {
    if (start == end) { tree[node] = A[start]; return; }   /* 叶子 */
    int mid = (start + end) / 2;
    Build(2*node,   start, mid);       /* 左孩子负责左半 */
    Build(2*node+1, mid+1, end);       /* 右孩子负责右半 */
    tree[node] = tree[2*node] + tree[2*node+1];  /* 回溯：合并左右区间和 */
}

/* 查询区间 [L, R] 的和 */
int Query(int node, int start, int end, int L, int R) {
    if (R < start || end < L) return 0;            /* 完全不相交：返回单位元 0 */
    if (L <= start && end <= R) return tree[node]; /* 完全包含：直接返回 */
    int mid = (start + end) / 2;                   /* 部分相交：递归左右后合并 */
    return Query(2*node, start, mid, L, R)
         + Query(2*node+1, mid+1, end, L, R);
}
```

```
21
22 /* 单点更新: 把 A[idx] 改为 val */
23 void Update(int node, int start, int end, int idx, int val) {
24     if (start == end) { tree[node] = val; return; }
25     int mid = (start + end) / 2;
26     if (idx <= mid) Update(2*node,   start, mid, idx, val);
27     else            Update(2*node+1, mid+1, end, idx, val);
28     tree[node] = tree[2*node] + tree[2*node+1];    /* 回溯更新祖先 */
29 }
```

4 优先队列与二叉堆（Priority Queue / Binary Heap）

以 min-heap 为例，下标从 1 开始（`Elements[0]` 作哨兵）。

结构定义

```
struct HeapStruct {
    int Capacity;
    int Size;              /* 当前元素个数 */
    int *Elements;         /* 下标 1..Size 存数据，Elements[0] 为哨兵 */
};
typedef struct HeapStruct *PriorityQueue;
```

插入 Insert（上滤 Percolate Up）

$T = O(\log N)$

```
void Insert(int X, PriorityQueue H) {
    if (H->Size == H->Capacity) return;      /* 堆满 */
    int i = ++H->Size;                       /* 新元素先放到末尾（保持完全二叉树）*/
    H->Elements[0] = X;                      /* 哨兵：当 i=1 时让循环自然停止 */
    for (; H->Elements[i/2] > X; i /= 2)     /* 父结点比 X 大就下移，X 上滤 */
        H->Elements[i] = H->Elements[i/2];   /* 用赋值代替交换，更快 */
    H->Elements[i] = X;                      /* 找到合适位置，放入 X */
}
```

删除最小 DeleteMin（下滤 Percolate Down）

$T = O(\log N)$

```
int DeleteMin(PriorityQueue H) {
    if (H->Size == 0) return H->Elements[0];   /* 空堆 */
    int MinElement = H->Elements[1];           /* 根即最小值 */
    int Last = H->Elements[H->Size--];         /* 取出末尾元素，准备填到根再下滤 */
    int i, Child;
    for (i = 1; i*2 <= H->Size; i = Child) {
        Child = i * 2;                         /* 左孩子 */
        if (Child != H->Size && H->Elements[Child+1] < H->Elements[Child])
            Child++;                           /* 选较小的孩子（右孩子存在且更小）*/
        if (Last > H->Elements[Child])         /* Last 比较小孩子大 -> 孩子上移 */
            H->Elements[i] = H->Elements[Child];
        else
            break;                             /* 找到位置，停止下滤 */
    }
    H->Elements[i] = Last;
    return MinElement;
}
```

建堆 BuildHeap（自底向上）

$T = O(N)$（不是 $O(N \log N)$！）

```
/* 元素已放入 Elements[1..N]，从最后一个非叶结点 N/2 起逆序下滤 */
```

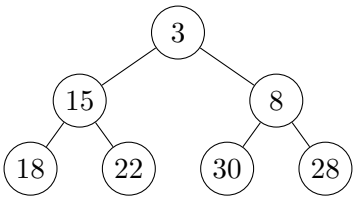
```
void BuildHeap(PriorityQueue H, int N) {
    H->Size = N;
    for (int i = N / 2; i >= 1; i--) {     /* 对每个非叶结点做一次下滤 */
        int Last = H->Elements[i], Child, j;
        for (j = i; j*2 <= N; j = Child) {
            Child = j * 2;
            if (Child != N && H->Elements[Child+1] < H->Elements[Child])
                Child++;
            if (Last > H->Elements[Child]) H->Elements[j] = H->Elements[Child];
            else break;
        }
        H->Elements[j] = Last;
    }
}
```

手工模拟：建堆 + DeleteMin（例 15 22 30 18 3 8 28，建 min-heap，下标 1 为根）

线性建堆：从最后一个非叶结点 $\lfloor 7/2 \rfloor = 3$ 起逆序下滤：

下滤结点	说明	数组（下标 1~7）
初始	—	15 22 30 18 3 8 28
$i=3$（30）	与较小孩子 8 交换	15 22 **8** 18 3 **30** 28
$i=2$（22）	与较小孩子 3 交换	15 **3** 8 18 **22** 30 28
$i=1$（15）	与较小孩子 3 交换	**3 15** 8 18 22 30 28

建成的 min-heap 树形：



连续 DeleteMin 两次（取根，末尾元素填根后下滤）：

操作	说明	数组状态
DeleteMin (1)	取走 3；28 填根后下滤	8 15 28 18 22 30
DeleteMin (2)	取走 8；30 填根后下滤	15 18 28 30 22

最终堆的层序为 **15 18 28 30 22**。

5 排序（Sorting）

以下均对 `int A[]` 升序排序，规模为 `N`。

插入排序 Insertion Sort

$T = O(N^2)$，最好 $O(N)$；稳定；$S = O(1)$

```c
void InsertionSort(int A[], int N) {
    for (int P = 1; P < N; P++) {      /* A[0..P-1] 已有序，把 A[P] 插入 */
        int Tmp = A[P];                /* 待插入元素 */
        int j;
        for (j = P; j > 0 && A[j-1] > Tmp; j--)
            A[j] = A[j-1];             /* 比 Tmp 大的元素整体后移 */
        A[j] = Tmp;                    /* 空出的位置放入 Tmp */
    }
}
```

手工模拟：例 `8 3 9 2 5`（方括号内为已排序前缀）

趟	操作	数组状态
初始	—	8 3 9 2 5
$P=1$	插入 3	[3 8] 9 2 5
$P=2$	插入 9	[3 8 9] 2 5
$P=3$	插入 2	[2 3 8 9] 5
$P=4$	插入 5	[2 3 5 8 9]

选择排序 Selection Sort

$T = O(N^2)$；不稳定；$S = O(1)$

```c
void SelectionSort(int A[], int N) {
    for (int i = 0; i < N - 1; i++) {
        int min = i;                   /* 在 A[i..N-1] 中找最小元素的下标 */
        for (int j = i + 1; j < N; j++)
            if (A[j] < A[min]) min = j;
        if (min != i) {                /* 把最小元素换到位置 i */
            int t = A[i]; A[i] = A[min]; A[min] = t;
        }
    }
}
```

冒泡排序 Bubble Sort

$T = O(N^2)$；稳定；$S = O(1)$

```c
void BubbleSort(int A[], int N) {
    for (int i = 0; i < N - 1; i++) {
        int swapped = 0;               /* 优化：某趟无交换说明已有序 */
        for (int j = 0; j < N-1-i; j++)      /* 每趟把最大元素「冒」到末尾 */
            if (A[j] > A[j+1]) {
```

```
                int t = A[j]; A[j] = A[j+1]; A[j+1] = t;
                swapped = 1;
            }
        if (!swapped) break;
    }
}
```

希尔排序 Shellsort

$T \approx O(N^{1.5})$（取决增量）；不稳定；$S = O(1)$

```
void Shellsort(int A[], int N) {
    /* 增量序列：N/2, N/4, ..., 1。每个增量做一次「间隔插入排序」 */
    for (int Inc = N / 2; Inc > 0; Inc /= 2)
        for (int i = Inc; i < N; i++) {
            int Tmp = A[i], j;
            for (j = i; j >= Inc && A[j-Inc] > Tmp; j -= Inc)
                A[j] = A[j-Inc];      /* 间隔为 Inc 的插入排序 */
            A[j] = Tmp;
        }
}
```

手工模拟：例 8 3 9 2 5 7 1，增量 $3 \to 1$（Inc=3 时下标差 3 的元素一组：$\{8,2,1\},\{3,5\},\{9,7\}$）

阶段	增量	数组状态
初始	—	8 3 9 2 5 7 1
第 1 趟	3	1 3 7 2 5 9 8
第 2 趟	1	1 2 3 5 7 8 9

堆排序 Heapsort（原地，下标从 0 开始）

$T = O(N \log N)$；不稳定；$S = O(1)$

```
#define LeftChild(i) (2*(i)+1)        /* 0-based：左孩子 = 2i+1 */

/* 下滤：在区间 [i, N) 上把 A[i] 调整到合适位置（最大堆） */
static void PercDown(int A[], int i, int N) {
    int Tmp = A[i], Child;
    for (; LeftChild(i) < N; i = Child) {
        Child = LeftChild(i);
        if (Child != N-1 && A[Child+1] > A[Child]) Child++; /* 取较大孩子 */
        if (Tmp < A[Child]) A[i] = A[Child];                /* 孩子更大则上移 */
        else break;
    }
    A[i] = Tmp;
}
void Heapsort(int A[], int N) {
    for (int i = N/2 - 1; i >= 0; i--)  /* 第一步：建最大堆 O(N) */
```

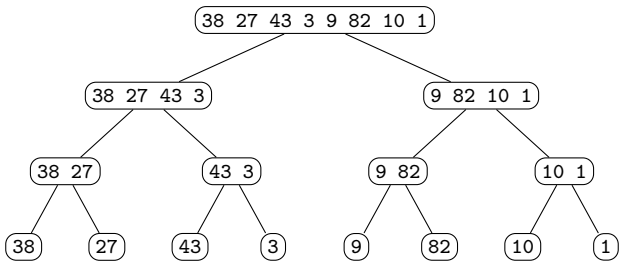
```
        PercDown(A, i, N);
    for (int i = N - 1; i > 0; i--) {   /* 第二步：反复把堆顶(最大)换到末尾 */
        int t = A[0]; A[0] = A[i]; A[i] = t;  /* 最大值归位 */
        PercDown(A, 0, i);              /* 对剩余 [0, i) 重新下滤 */
    }
}
```

归并排序 Mergesort（递归）

$T = O(N \log N)$；稳定；$S = O(N)$

```
/* 合并两个有序段 A[L..M] 与 A[M+1..R] 到临时数组再写回 */
static void Merge(int A[], int Tmp[], int L, int M, int R) {
    int i = L, j = M + 1, k = L;
    while (i <= M && j <= R)
        Tmp[k++] = (A[i] <= A[j]) ? A[i++] : A[j++]; /* <= 保证稳定 */
    while (i <= M) Tmp[k++] = A[i++];   /* 接上左段剩余 */
    while (j <= R) Tmp[k++] = A[j++];   /* 接上右段剩余 */
    for (k = L; k <= R; k++) A[k] = Tmp[k];          /* 写回原数组 */
}
static void MSort(int A[], int Tmp[], int L, int R) {
    if (L >= R) return;                 /* 单元素或空，已有序 */
    int M = (L + R) / 2;
    MSort(A, Tmp, L, M);                /* 排左半 */
    MSort(A, Tmp, M+1, R);              /* 排右半 */
    Merge(A, Tmp, L, M, R);            /* 合并 */
}
void Mergesort(int A[], int N) {
    int *Tmp = (int*)malloc(N * sizeof(int));   /* 只分配一次临时数组 */
    MSort(A, Tmp, 0, N - 1);
    free(Tmp);
}
```

手工模拟：递归树（例 38 27 43 3 9 82 10 1）——不断二分到单元素，再两两有序合并



合并（自底向上）：

- 层 1：$[38,27] \to [27,38]$，$[43,3] \to [3,43]$，$[9,82]$，$[10,1] \to [1,10]$
- 层 2：$[27,38]$ 与 $[3,43]$ 合为 $[3,27,38,43]$；右半合为 $[1,9,10,82]$
- 层 3：$[3,27,38,43]$ 与 $[1,9,10,82]$ 合为 $[1,3,9,10,27,38,43,82]$

归并排序（迭代 / 自底向上）

$T = O(N \log N)$；稳定；$S = O(N)$

```c
void MergesortIter(int A[], int N) {
    int *Tmp = (int*)malloc(N * sizeof(int));
    for (int len = 1; len < N; len *= 2) {        /* 子段长度 1,2,4,8,... */
        for (int L = 0; L + len < N; L += 2*len) { /* 两两相邻子段合并 */
            int M = L + len - 1;
            int R = (L + 2*len - 1 < N) ? L + 2*len - 1 : N - 1; /* 防越界 */
            Merge(A, Tmp, L, M, R);
        }
    }
    free(Tmp);
}
```

手工模拟：例 25 21 47 15 27 68 35 20，段长从 1 翻倍（竖线分隔有序段）

趟	段长	数组状态
初始	1	25 21 | 47 15 | 27 68 | 35 20
第 1 趟	2	21 25 | 15 47 | 27 68 | 20 35
第 2 趟	4	15 21 25 47 | 20 27 35 68
第 3 趟	8	15 20 21 25 27 35 47 68

快速排序 Quicksort（三数取中 + 小数组用插入排序）

平均 $T = O(N \log N)$，最坏 $O(N^2)$；不稳定

```c
#define Cutoff 10
static void Swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

/* 三数取中：把 A[Left],A[Center],A[Right] 排序，取中值作 pivot 并藏到 Right-1 */
static int Median3(int A[], int Left, int Right) {
    int Center = (Left + Right) / 2;
    if (A[Left] > A[Center])  Swap(&A[Left], &A[Center]);
    if (A[Left] > A[Right])   Swap(&A[Left], &A[Right]);
    if (A[Center] > A[Right]) Swap(&A[Center], &A[Right]);
    Swap(&A[Center], &A[Right-1]);      /* 把 pivot 放到 Right-1 */
    return A[Right-1];                  /* 返回 pivot 值 */
}
void Qsort(int A[], int Left, int Right) {
    if (Left + Cutoff <= Right) {       /* 大数组才用快排 */
        int Pivot = Median3(A, Left, Right);
        int i = Left, j = Right - 1;
        for (;;) {
            while (A[++i] < Pivot) {}   /* i 右移直到遇到 >= pivot 的元素 */
            while (A[--j] > Pivot) {}   /* j 左移直到遇到 <= pivot 的元素 */
            if (i < j) Swap(&A[i], &A[j]);  /* 交换这对逆序元素 */
```

```
            else break;                 /* 指针相遇，分区结束 */
        }
        Swap(&A[i], &A[Right-1]);       /* 把 pivot 换到分界位置 i */
        Qsort(A, Left, i - 1);          /* 递归左半 */
        Qsort(A, i + 1, Right);         /* 递归右半 */
    } else
        InsertionSortRange(A, Left, Right); /* 小数组：插入排序更快 */
}
/* 初始调用：Qsort(A, 0, N-1);  (InsertionSortRange 为对 A[Left..Right] 的插入排序) */
```

手工模拟：一次划分（三数取中，例 8 1 4 9 6 3 5 2 7 0）

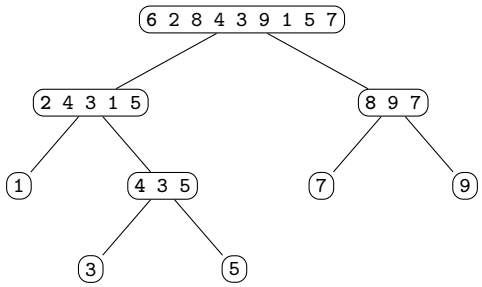
Left=8, *Right*=0, *Center*=3，三数取中后 pivot= 3 藏到 *Right*−1；竖线标出 pivot：

步骤	数组状态
取中后藏 pivot	0 1 4 9 6 7 5 2 |3| 8
i 停于 9、j 停于 2，交换	0 1 2 9 6 7 5 4 |3| 8
i,j 交错，pivot 归位	0 1 2 **3** 6 7 5 4 9 8

pivot 3 落入最终位置，左侧全 <3、右侧全 >3，再递归两侧。

手工模拟：递归树（例 6 2 8 4 3 9 1 5 7，为清晰起见以**首元素**为 pivot）

每个结点为待排子数组，**加粗**为 pivot，左右子结点为划分出的两段：



最坏情况（每次 pivot 恰为最大 / 最小）退化为深度 N 的链，故 $O(N^2)$；三数取中可大概率避免。

快速选择 Quickselect（求第 k 小）

平均 $T = O(N)$

```
/* 找 A[Left..Right] 中第 k 小（k 从 1 计）。思想同快排但只递归一侧 */
int Qselect(int A[], int k, int Left, int Right) {
    if (Left + Cutoff <= Right) {
        int Pivot = Median3(A, Left, Right);
        int i = Left, j = Right - 1;
        for (;;) {
            while (A[++i] < Pivot) {}
            while (A[--j] > Pivot) {}
            if (i < j) Swap(&A[i], &A[j]);
            else break;
        }
```

```
        Swap(&A[i], &A[Right-1]);
        if (k <= i - Left + 1)   return Qselect(A, k, Left, i-1);     /* 第 k 小在左侧 */
        else if (k > i - Left + 1) return Qselect(A, k - (i-Left+1), i+1, Right); /* 在右侧 */
        else return A[i];        /* pivot 恰为第 k 小 */
    } else {
        InsertionSortRange(A, Left, Right);
        return A[Left + k - 1];
    }
}
```

基数排序 Radix Sort（LSD，非负整数）

$T = O(d(N + r))$；稳定

```
/* 从最低位(个位)到最高位，依次按当前位做「计数/桶」分配收集 */
void RadixSort(int A[], int N) {
    int max = A[0];
    for (int i = 1; i < N; i++) if (A[i] > max) max = A[i]; /* 求最大值定位数 */
    int *out = (int*)malloc(N * sizeof(int));
    for (int exp = 1; max / exp > 0; exp *= 10) {  /* exp = 1,10,100,... 当前位 */
        int count[10] = {0};
        for (int i = 0; i < N; i++)
            count[(A[i]/exp) % 10]++;              /* 统计每个数字(0~9)出现次数 */
        for (int d = 1; d < 10; d++)
            count[d] += count[d-1];                /* 前缀和 -> 每个桶的结束位置 */
        for (int i = N - 1; i >= 0; i--) {         /* 从后往前保证稳定 */
            int d = (A[i]/exp) % 10;
            out[--count[d]] = A[i];                /* 放到对应桶的正确位置 */
        }
        for (int i = 0; i < N; i++) A[i] = out[i]; /* 写回，进入下一位 */
    }
    free(out);
}
```

6 散列表（Hashing）

6.1 分离链接法 Separate Chaining

结构、查找、插入

平均 $T = O(1+\lambda)$，λ 为装载因子

```c
struct ListNode { int Element; struct ListNode *Next; };
typedef struct ListNode *Position;
struct HashTbl {
    int TableSize;
    Position *TheLists;   /* 指针数组：每个桶是一条链表（带头结点）*/
};
typedef struct HashTbl *HashTable;

int Hash(int Key, int TableSize) {
    return Key % TableSize;          /* 整数键：取模；TableSize 应为素数 */
}

/* 查找：定位到桶链表后顺序查找 */
Position Find(int Key, HashTable H) {
    Position P = H->TheLists[Hash(Key, H->TableSize)]->Next; /* 跳过头结点 */
    while (P != NULL && P->Element != Key)
        P = P->Next;
    return P;                        /* 未找到返回 NULL */
}

/* 插入：若不存在则头插到对应链表 */
void Insert(int Key, HashTable H) {
    if (Find(Key, H) != NULL) return;        /* 已存在，不重复插 */
    Position L = H->TheLists[Hash(Key, H->TableSize)];
    Position New = (Position)malloc(sizeof(struct ListNode));
    New->Element = Key;
    New->Next = L->Next;             /* 头插：接到头结点之后 */
    L->Next = New;
}
```

6.2 开放地址法 Open Addressing（平方探测 Quadratic Probing）

结构、查找、插入

$\lambda < 0.5$ 且表长为素数时保证可插入

```c
enum Kind { Legitimate, Empty, Deleted };  /* 槽的三种状态 */
struct HashEntry { int Element; enum Kind Info; };
struct HashTbl2 { int TableSize; struct HashEntry *Cells; };
typedef struct HashTbl2 *HashTable2;

```

```c
/* 平方探测查找：冲突位置序列 h, h+1, h+4, h+9, ... (mod TableSize) */
int Find2(int Key, HashTable2 H) {
    int Pos = Hash(Key, H->TableSize);
    int Collision = 0;
    while (H->Cells[Pos].Info != Empty && H->Cells[Pos].Element != Key) {
        Pos += 2 * ++Collision - 1;   /* 增量递推：f(i)=f(i-1)+2i-1，免去平方运算 */
        if (Pos >= H->TableSize)
            Pos -= H->TableSize;      /* 用减法代替取模 */
        /* 注意：遇到 Deleted 必须继续探测，不能停！ */
    }
    return Pos;                       /* 返回 Key 所在位置，或第一个 Empty 位置 */
}

void Insert2(int Key, HashTable2 H) {
    int Pos = Find2(Key, H);
    if (H->Cells[Pos].Info != Legitimate) {  /* 该位置为空或已删，可放入 */
        H->Cells[Pos].Info = Legitimate;
        H->Cells[Pos].Element = Key;
    }
}
```

手工模拟：平方探测插入（表长 13，$H(k) = k \bmod 13$，插入 20 6 2 16 27 15）

探测序列 $H(k), H(k)+1^2, H(k)+2^2, \ldots \pmod{13}$：

键	$H(k)$	探测过程 → 落点
20	7	7 空 → **7**
6	6	6 空 → **6**
2	2	2 空 → **2**
16	3	3 空 → **3**
27	1	1 空 → **1**
15	2	2(占)→ 3(占)→ 2+4=6(占)→ 2+9=11 空 → **11**

最终表：下标 $1, 2, 3, 6, 7, 11$ 处分别为 $27, 2, 16, 6, 20, 15$。

再散列 Rehash

单次 $O(N)$，摊还 $O(1)$

```c
/* 当装载因子超限或插入失败时调用：建约 2 倍大的素数表，重新插入所有有效元素 */
HashTable2 Rehash(HashTable2 H) {
    int OldSize = H->TableSize;
    struct HashEntry *OldCells = H->Cells;
    HashTable2 New = InitializeTable(NextPrime(2 * OldSize)); /* 新表：下一个素数 */
    for (int i = 0; i < OldSize; i++)
        if (OldCells[i].Info == Legitimate)  /* 只迁移有效元素，跳过 Empty/Deleted */
            Insert2(OldCells[i].Element, New);
```

```
9       free(OldCells);
10      return New;
11  }
```

7 并查集（Disjoint Set / Union-Find）

用数组 S[] 表示森林：S[x] 为父结点下标；根处存 − 集合大小（或负的高度/秩）。

初始化

每个元素自成一个集合

```
int S[MAXN];
void Initialize(int N) {
    for (int i = 0; i < N; i++) S[i] = -1;  /* -1 表示：自身是根，集合大小为 1 */
}
```

查找 Find（路径压缩，递归版）

均摊近似 $O(1)$

```
int Find(int X) {
    if (S[X] < 0) return X;          /* 负值表示根，返回自身 */
    return S[X] = Find(S[X]);        /* 路径压缩：把 X 直接挂到根下，并返回根 */
}
```

查找 Find（路径压缩，迭代版）

均摊近似 $O(1)$

```
int FindIter(int X) {
    int root = X;
    while (S[root] >= 0) root = S[root];     /* 第一遍：找到根 */
    while (S[X] >= 0) {                       /* 第二遍：把路径上结点都挂到根 */
        int next = S[X];
        S[X] = root;
        X = next;
    }
    return root;
}
```

合并 Union（按大小合并 Union-by-Size）

保证树高 $O(\log N)$

```
/* 传入的是两个根 Root1, Root2。把小树挂到大树下 */
void SetUnion(int Root1, int Root2) {
    if (Root1 == Root2) return;
    /* S[root] = -size，值越小（越负）集合越大 */
    if (S[Root2] < S[Root1]) {        /* Root2 所在集合更大 */
        S[Root2] += S[Root1];         /* 累加大小 */
        S[Root1] = Root2;             /* Root1 挂到 Root2 下 */
    } else {                          /* Root1 不小于 Root2 */
        S[Root1] += S[Root2];
        S[Root2] = Root1;
    }
}
/* 按高度/秩合并：把矮树挂到高树下，等高时合并后高度 +1，原理类似 */
```

手工模拟：Union-by-Size + 路径压缩（元素 0∼7，等大时第一个根作根）

依次 `union(find(1),find(5))`、`(3,6)`、`(0,1)`、`(6,5)`、`(6,4)`；`S[]` 初始全 -1（根存 $-$ 大小）：

操作后	S[0]	S[1]	S[2]	S[3]	S[4]	S[5]	S[6]	S[7]
初始	-1	-1	-1	-1	-1	-1	-1	-1
union(1,5)	-1	-2	-1	-1	-1	1	-1	-1
union(3,6)	-1	-2	-1	-2	-1	1	3	-1
union(0,1)	1	-3	-1	-2	-1	1	3	-1
union(6,5)	1	-5	-1	1	-1	1	3	-1
union(6,4)	1	-6	-1	1	1	1	1	-1

最终元素 1 为根（集合 $\{0,1,3,4,5,6\}$，大小 6），元素 2、7 各自独立。

8 图（Graph Algorithms）

统一用**邻接矩阵 (adjacency matrix)** 表示：`G[v][w]` 为边权，无边记为 `INF`（无权图用 0/1）。

结构定义

邻接矩阵 $O(V^2)$ 空间

```
#define MaxV 100
#define INF  1000000
typedef struct {
    int Nv, Ne;             /* 顶点数、边数 */
    int G[MaxV][MaxV];      /* 邻接矩阵；G[v][w] = 权值或 INF */
} MGraph;
```

8.1 拓扑排序 Topological Sort

基于入度 + 队列（Kahn 算法）

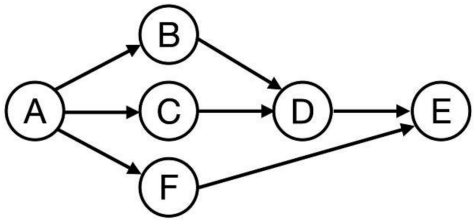
$T = O(V^2)$（矩阵）或 $O(V+E)$（邻接表）

```
/* 把拓扑序写入 order[]；若存在环则返回 0（无法完成） */
int TopSort(MGraph *Gr, int order[]) {
    int indeg[MaxV] = {0};
    for (int v = 0; v < Gr->Nv; v++)          /* 计算每个顶点的入度 */
        for (int w = 0; w < Gr->Nv; w++)
            if (Gr->G[v][w] != INF && v != w) indeg[w]++;

    int queue[MaxV], front = 0, rear = 0, cnt = 0;
    for (int v = 0; v < Gr->Nv; v++)
        if (indeg[v] == 0) queue[rear++] = v;     /* 入度为 0 的先入队 */

    while (front < rear) {
        int v = queue[front++];                  /* 取出一个入度 0 顶点 */
        order[cnt++] = v;                         /* 输出到拓扑序 */
        for (int w = 0; w < Gr->Nv; w++)          /* 删除 v 的出边 */
            if (Gr->G[v][w] != INF && v != w)
                if (--indeg[w] == 0)              /* 邻接点入度减 1，变 0 则入队 */
                    queue[rear++] = w;
    }
    return cnt == Gr->Nv;                         /* 全部输出则无环；否则有环 */
}
```

手工模拟：拓扑排序（入度为 0 入队，按字母序处理）



边：$A\to B$, $A\to C$, $A\to F$, $B\to D$, $C\to D$, $D\to E$, $F\to E$。各点入度随出队动态变化：

步	输出	A	B	C	D	E	F	队列
0	—	0	1	1	2	2	1	A
1	A	–	0	0	2	2	0	B C F
2	B	–	–	0	1	2	0	C F
3	C	–	–	–	0	2	0	F D
4	F	–	–	–	0	1	–	D
5	D	–	–	–	–	0	–	E
6	E	–	–	–	–	–	–	（空）

一个拓扑序：**A B C F D E**（该图共有 8 个不同拓扑序）。

8.2　无权最短路 Unweighted Shortest Path（BFS）

从源点 S 出发的 BFS

$T = O(V^2)$（矩阵）

```
void Unweighted(MGraph *Gr, int S, int dist[]) {
    for (int i = 0; i < Gr->Nv; i++) dist[i] = -1; /* -1 表示未访问 */
    int queue[MaxV], front = 0, rear = 0;
    dist[S] = 0; queue[rear++] = S;            /* 源点距离 0, 入队 */
    while (front < rear) {
        int v = queue[front++];
        for (int w = 0; w < Gr->Nv; w++)
            if (Gr->G[v][w] != INF && dist[w] == -1) { /* w 相邻且未访问 */
                dist[w] = dist[v] + 1;         /* 层数 +1 即最短边数 */
                queue[rear++] = w;
            }
    }
}
```

8.3　Dijkstra 单源最短路（非负权）

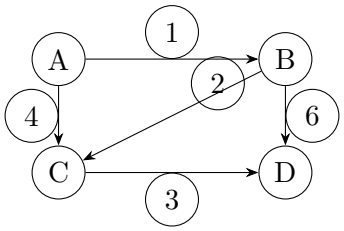
邻接矩阵实现

$T = O(V^2)$；用堆可优化到 $O(E\log V)$

```
void Dijkstra(MGraph *Gr, int S, int dist[]) {
```

```
    int known[MaxV] = {0};
    for (int i = 0; i < Gr->Nv; i++) dist[i] = INF;
    dist[S] = 0;
    for (int iter = 0; iter < Gr->Nv; iter++) {
        /* 1) 在未确定(known=0)的顶点里选 dist 最小者 */
        int v = -1, min = INF;
        for (int i = 0; i < Gr->Nv; i++)
            if (!known[i] && dist[i] < min) { min = dist[i]; v = i; }
        if (v == -1) break;                     /* 剩余都不可达 */
        known[v] = 1;                           /* v 的最短路就此确定 */
        /* 2) 用 v 松弛(relax)其邻接点 */
        for (int w = 0; w < Gr->Nv; w++)
            if (!known[w] && Gr->G[v][w] != INF
                && dist[v] + Gr->G[v][w] < dist[w])
                dist[w] = dist[v] + Gr->G[v][w]; /* 经 v 到 w 更短则更新 */
    }
}
/* 警告：图中有负权边时 Dijkstra 不再正确 */
```

手工模拟：Dijkstra（源点 A）——逐步确定与松弛



每次从未确定顶点中选 `dist` 最小者（**加粗**）确定，并松弛其邻接点：

步	确定	dist[A]	dist[B]	dist[C]	dist[D]
初始	—	0	∞	∞	∞
1	**A**	0	1	4	∞
2	**B**	0	1	3	7
3	**C**	0	1	3	6
4	**D**	0	1	3	6

确定顺序 **A, B, C, D**；最短距离 A=0, B=1, C=3, D=6。

8.4　SPFA 单源最短路（可含负权）

Bellman–Ford 的队列优化：只有 dist 被改小的点才重新入队去松弛邻居　　平均 $O(kE)$，最坏 $O(VE)$；可处理负权边并检测负环

```c
void SPFA(MGraph *Gr, int S, int dist[]) {
    int inQueue[MaxV], cnt[MaxV];           /* inQueue: 是否在队中; cnt: 入队次数 */
    int queue[MaxV], front = 0, rear = 0;   /* 简单数组队列 */
    for (int i = 0; i < Gr->Nv; i++) {
        dist[i] = INF;
        inQueue[i] = 0;
        cnt[i] = 0;
    }
    dist[S] = 0;
    queue[rear++] = S;                      /* 源点入队 */
    inQueue[S] = 1;
    cnt[S] = 1;                             /* 源点已入队一次 */
    while (front < rear) {
        int v = queue[front++];             /* 取出队首 */
        inQueue[v] = 0;                     /* 出队后清标记，允许它将来再次入队 */
        for (int w = 0; w < Gr->Nv; w++) {
            /* 经 v 到 w 更短则松弛 */
            if (Gr->G[v][w] != INF && dist[v] + Gr->G[v][w] < dist[w]) {
                dist[w] = dist[v] + Gr->G[v][w];   /* 先更新 dist[w] */
                if (!inQueue[w]) {          /* w 不在队中才入队，避免重复 */
                    queue[rear++] = w;
                    inQueue[w] = 1;
                    cnt[w]++;
                    if (cnt[w] >= Gr->Nv) return; /* 某点入队 >= V 次 => 存在负环 */
                }
                /* 若 w 已在队中：不重复入队，但 dist[w] 已改小，
                   等它出队时自然用最新值去松弛邻居 */
            }
        }
    }
}
```

- 与 Dijkstra 相比：Dijkstra 每点只确定一次、不能有负权；SPFA 允许一个点被反复松弛、可处理负权边。
- `cnt[w] >= Gr->Nv`：在 V 个顶点的图中，无负环时任意点最多入队 $V-1$ 次；超过即说明存在**负环 (negative cycle)**。

8.5　深度优先搜索 DFS 与连通分量

DFS（递归）

$T = O(V^2)$（矩阵）

```c
int visited[MaxV];
void DFS(MGraph *Gr, int v) {
```

```
    visited[v] = 1;                    /* 标记已访问 */
    visit(v);
    for (int w = 0; w < Gr->Nv; w++)
        if (Gr->G[v][w] != INF && !visited[w]) /* 沿未访问的邻接点深入 */
            DFS(Gr, w);
}
/* 统计连通分量个数：对每个未访问顶点各发起一次 DFS */
int CountComponents(MGraph *Gr) {
    for (int i = 0; i < Gr->Nv; i++) visited[i] = 0;
    int cnt = 0;
    for (int v = 0; v < Gr->Nv; v++)
        if (!visited[v]) { DFS(Gr, v); cnt++; } /* 每发起一次即一个新分量 */
    return cnt;
}
```

8.6　最小生成树 MST：Prim 算法

从一个顶点开始“长树”，每次并入最近的顶点

$T = O(V^2)$，适合稠密图

```
/* dist[w]：w 到「当前生成树」的最短边权；parent[w]：对应的树内端点 */
int Prim(MGraph *Gr, int parent[]) {
    int dist[MaxV], known[MaxV] = {0}, total = 0;
    for (int i = 0; i < Gr->Nv; i++) { dist[i] = INF; parent[i] = -1; }
    dist[0] = 0;                                   /* 从顶点 0 开始 */
    for (int iter = 0; iter < Gr->Nv; iter++) {
        int v = -1, min = INF;                     /* 选树外、dist 最小的顶点 */
        for (int i = 0; i < Gr->Nv; i++)
            if (!known[i] && dist[i] < min) { min = dist[i]; v = i; }
        if (v == -1) break;                        /* 图不连通 */
        known[v] = 1; total += dist[v];            /* v 并入生成树，累加权值 */
        for (int w = 0; w < Gr->Nv; w++)           /* 用 v 更新其邻接点到树的距离 */
            if (!known[w] && Gr->G[v][w] < dist[w]) {
                dist[w] = Gr->G[v][w];
                parent[w] = v;
            }
    }
    return total;                                  /* 返回 MST 总权值 */
}
```

8.7　最小生成树 MST：Kruskal 算法

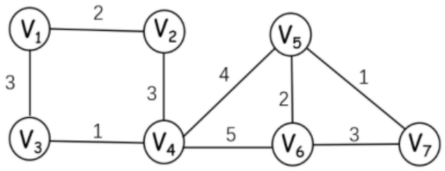
按边权升序选边，用并查集判环

$T = O(E \log E)$，适合稀疏图

```
struct Edge { int u, v, w; };                      /* 边：两端点 + 权值 */
```

```c
int cmp(const void *a, const void *b) {        /* 按权值升序 */
    return ((struct Edge*)a)->w - ((struct Edge*)b)->w;
}
/* edges[0..m-1] 为全部边，n 为顶点数；S[] 为并查集，用前面的 Find/SetUnion */
int Kruskal(struct Edge edges[], int m, int n) {
    qsort(edges, m, sizeof(struct Edge), cmp); /* 1) 所有边按权排序 */
    Initialize(n);                             /* 2) 并查集初始化 */
    int total = 0, count = 0;
    for (int i = 0; i < m && count < n - 1; i++) {
        int ru = Find(edges[i].u), rv = Find(edges[i].v);
        if (ru != rv) {                        /* 3) 两端不在同一集合 -> 选这条边 */
            SetUnion(ru, rv);                  /*    合并，避免成环 */
            total += edges[i].w;
            count++;                           /* 已选 n-1 条边即完成 */
        }
    }
    return total;                              /* 返回 MST 总权值 */
}
```

手工模拟：Kruskal 与 Prim（同一带权图）



Kruskal（边按权升序，用并查集判环；能连通新分量则选，否则弃）：

边	权	取舍	说明
(v_3, v_4)	1	选	新连通
(v_5, v_7)	1	选	新连通
(v_1, v_2)	2	选	新连通
(v_5, v_6)	2	选	新连通
(v_1, v_3)	3	选	合并 $\{1,2\}$ 与 $\{3,4\}$
(v_2, v_4)	3	弃	成环
(v_6, v_7)	3	弃	成环
(v_4, v_5)	4	选	合并两大集合，共 6 条边，结束

Prim（从 v_1 出发，每步并入与树相连的最小边端点）依次并入：$v_1 \xrightarrow{2} v_2 \xrightarrow{3} v_3 \xrightarrow{1} v_4 \xrightarrow{4} v_5 \xrightarrow{1} v_7 \xrightarrow{2} v_6$。两法总权值相同：$1+1+2+2+3+4 = \mathbf{13}$。

8.8　网络最大流 Maximum Flow（Edmonds–Karp）

反复用 BFS 找增广路，在残量图上增流

$T = O(VE^2)$

```
int cap[MaxV][MaxV];      /* 残量容量矩阵：cap[u][v] 为 u->v 当前可增流量 */

/* 用 BFS 找一条从 s 到 t 的增广路，path[] 记录每个点的前驱；返回是否找到 */
int BFS(int n, int s, int t, int path[]) {
    int visited[MaxV] = {0};
    int queue[MaxV], front = 0, rear = 0;
    visited[s] = 1; path[s] = -1; queue[rear++] = s;
    while (front < rear) {
        int u = queue[front++];
        for (int v = 0; v < n; v++)
            if (!visited[v] && cap[u][v] > 0) { /* 仍有剩余容量的边可走 */
                visited[v] = 1; path[v] = u; queue[rear++] = v;
                if (v == t) return 1;           /* 到达汇点，找到增广路 */
            }
    }
    return 0;                                   /* 找不到增广路 */
}
int MaxFlow(int n, int s, int t) {
    int path[MaxV], flow = 0;
    while (BFS(n, s, t, path)) {
        int aug = INF;                          /* 1) 求增广路上的瓶颈(最小残量) */
        for (int v = t; v != s; v = path[v])
            if (cap[path[v]][v] < aug) aug = cap[path[v]][v];
        for (int v = t; v != s; v = path[v]) {  /* 2) 沿路增流 */
            cap[path[v]][v] -= aug;             /*    正向边减少残量 */
            cap[v][path[v]] += aug;             /*    反向边增加残量（允许“反悔”） */
        }
        flow += aug;                            /* 3) 累加本次增广量 */
    }
    return flow;                                /* 最大流 = 最小割 */
}
```

——讲义完——配合《期末复习讲义》使用，效果更佳！